



TEMA: SISTEMAS EXPERTOS (PROLOG)

1.- OBJETIVOS

Al finalizar la práctica el estudiante estará en condiciones de:

- Implementar algoritmos con el paradigma declarativo en el lenguaje de programación PROLOG.
- Implementar bases de conocimientos (Hechos y reglas) en la herramienta SWI Prolog.
- Implementar un sistema experto.

2.- TRABAJO PREPARATORIO

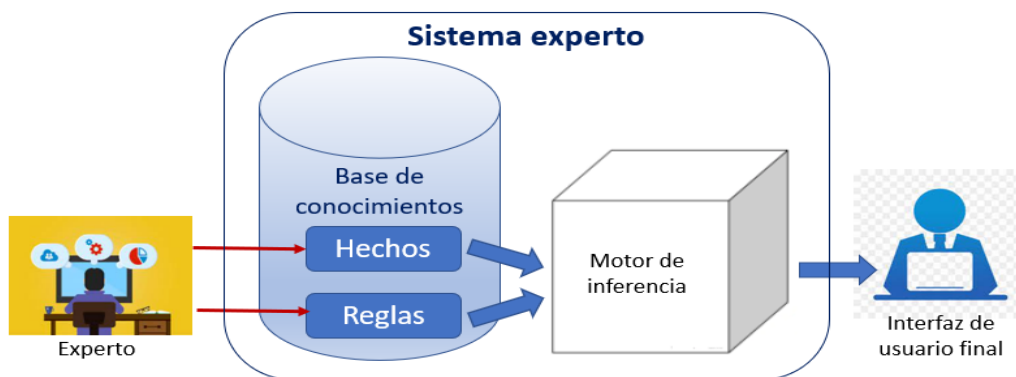
- Descargar e instalar el software SWI Prolog.
- Estudiar los conceptos básicos del lenguaje de programación PROLOG.

3.- BASE TEORICA COMPLEMENTARIA

Sistema experto

"Un sistema informático que simula el proceso de aprendizaje, de memorización, de razonamiento, de comunicación y de acción de un experto humano en una determinada rama de la ciencia o campo, suministrando, de esta forma, un consultor que puede sustituirle con unas ciertas garantías de éxito".

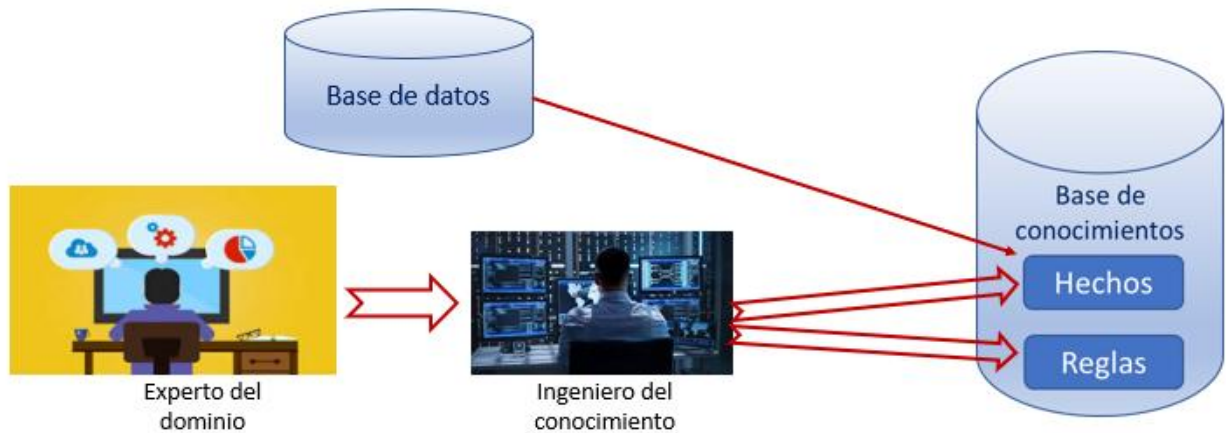
Arquitectura de un sistema experto



Base de conocimientos basado en reglas

La base de conocimientos de un sistema experto está constituido por “hechos” y “reglas”.

- ☐ Los hechos se pueden obtener de una base de datos y/o del experto del dominio.
- ☐ Las reglas las determina el experto del dominio, las que son formalizadas por el ingeniero del conocimiento y almacenadas en la base de conocimientos.



Motor de inferencia

El motor de inferencia es el módulo que activa, controla y ejecuta el razonamiento que se efectúa sobre el conocimiento que en forma de hechos y reglas se halla implementado en la base de conocimientos para poder alcanzar nuevos conocimientos. Tiene como tareas fundamentales gestionar el conocimiento y obtener conclusiones. Es el componente que empleará la base de conocimiento para responder las consultas de los usuarios. Es el programa “supervisor” que está entre el usuario y la base de conocimientos, y que extrae conclusiones a partir de los datos simbólicos que están almacenados en las bases de hechos.

Las estrategias o lógica de inferencia del motor, se implementa en base a dos tipos de algoritmos:

- ☐ Encadenamiento hacia delante (forward chaining, progresivo o deductivo)
- ☐ Encadenamiento hacia atrás (backward chaining, regresivo o inductivo)

Lenguaje de programación PROLOG

Como shell para la programación de Sistemas Expertos Basados en Reglas, PROLOG usa Lógica de Predicados de Primer Orden (restringida a cláusulas de Horn) para representar datos y conocimiento, utiliza encadenamiento hacia atrás y una estrategia de control retroactiva sin información heurística (backtracking).

4.- CONTENIDO DE LA PRÁCTICA.

PARTE 1.- Implementación de predicados dinámicos

En SWI-Prolog, un predicado dinámico es un predicado cuyas cláusulas pueden ser modificadas durante la ejecución del programa. Permite agregar, eliminar o modificar hechos y reglas en tiempo de ejecución, lo que proporciona una capacidad de manipulación dinámica de la base de conocimientos.

En este ejercicio se ilustrará cómo manipular hechos dinámicamente.

Ejercicio 1.- Programa para manipular hechos dinámicamente.

- Abrir una nueva ventana para implementar el sistema experto.
- Copiar y pegar el siguiente código.

```
/* ***** HECHOS ***** */
% Personas
persona(ana, 13, femenino).
persona(joshua, 21, masculino).
persona(luis, 22, masculino).
persona(jorge, 25, masculino).
persona(isaac, 14, masculino).
persona(eva, 17, femenino).
persona(cecil, 16, femenino).
persona(david, 21, masculino).
persona(rocio, 19, femenino).
persona(juan, 12, masculino).
persona(pilar, 16, femenino).

/* ***** HECHOS GENERADOS DINÁMICAMENTE ***** */
% -- Declarar un predicado dinámico
:- dynamic menor_edad/3.

% Agregar personas cuya edad es menor a 18 al predicado dinámico menor_edad/3
agregar_menor_edad :-
    % -- Limpiar el predicado dinámico
    retractall(menor_edad(_, _, _)),
    % -- Agregar personas al predicado dinámico
    persona(Nombre, Edad, Genero),
    Edad < 18,
    % -- Agregar persona al inicio del predicado "menor_edad"
    asserta(menor_edad(Nombre, Edad, Genero)),
    % -- Para continuar con la siguiente persona, forzar una unificación no válida
    fail.
agregar_menor_edad.

/* ***** REGLAS ***** */
% menor de edad masculino
menor_edad_masculino(Nombre, Edad) :- menor_edad(Nombre, Edad, masculino).
% menor de edad femenino
menor_edad_femenino(Nombre, Edad) :- menor_edad(Nombre, Edad, femenino).
```

- Almacenar y compilar.
- Ejecutar los siguientes comandos:
 - Generar dinámicamente los **Hechos** en el predicado dinámico **menor_edad**, ejecutando:
`?- agregar_menor_edad.`
 - Mostrar la relación de **Hechos** generados dinámicamente, ejecutando:
`?- listing(menor_edad).`
 - Inferir los menores de edad del género femenino, ejecutando:
`?- menor_edad_femenino(N,E).`

Notar que el proceso de inferencia se realiza sobre los **Hechos menor_edad**.

PARTE 2.- Implementación de un sistema experto

En esta sección se implementará un sistema experto. La explicación del código estará a cargo del docente.

Ejercicio 2.- Programa para diagnosticar una enfermedad.

- Abrir una nueva ventana para implementar el sistema experto.
- Copiar y pegar el siguiente código.

```
/*
SISTEMA EXPERTO DE DIAGNÓSTICO MÉDICO
*/
/*
Base de conocimientos:
Hechos: Enfermedad, lista de síntomas.
*/

conocimiento('covid',
    ['pérdida del gusto o del olfato',
     'dificultad para respirar o falta de aire',
     'tos',
     'fiebre',
     'cansancio',
     'dolores musculares',
     'dolor de garganta y/o cabeza'
    ]).

conocimiento('viruela del mono',
    ['sarpullido con apariencia de granos o ampollas',
     'inflamación de los ganglios',
     'fiebre',
     'dolores musculares',
     'agotamiento',
     'escalofríos'
    ]).

conocimiento('gripe',
    ['malestar en todo el cuerpo',
     'fiebre',
     'dolor de cabeza'
    ]).

conocimiento('influenza',
    ['dolor en las articulaciones',
     'mucho estornudo',
     'dolor de cabeza'
    ]).

conocimiento('sarampion',
    ['cubierto el cuerpo de puntos',
     'los ojos rojos',
     'tos seca',
     'fiebre'
    ]).

conocimiento('malaria',
    ['temblor violento',
     'dolor en las articulaciones',
     'escalofríos',
     'fiebre'
    ]).

conocimiento('tifoidea',
    ['dolor abdominal',
     'falta de apetito',
     'diarrea',
     'fiebre',
     'dolor de cabeza'
    ]).

conocimiento('enfermedad desconocida', []).
```

```

/*
Reglas de la base de conocimientos.
*/

/* Fase 0: Inicializar variables dinámicas */
:- dynamic respuestas_afirmativas/1.
:- dynamic respuestas_negativas/1.

mensajes_previos :-
    write('*****'),nl,
    write('Por favor responda las siguientes preguntas'),nl,
    write('con <si> <no> o <porque>'),nl,
    write('*****'),nl.

/* Fase 1: Recuperar enfermedad y sintomas*/
sintomas(Diagnosis, Sintomas) :-
    conocimiento(Diagnosis, Sintomas).

/* Fase 2: Efectuar preguntas al paciente*/

/* Registrar que el síntoma ya fue contestado */
procesar_rpta(_, Sintoma, Rpta) :-
    Rpta == si,
    asserta(respuestas_afirmativas(Sintoma)).
procesar_rpta(_, Sintoma, Rpta) :-
    Rpta == no,
    asserta(respuestas_negativas(Sintoma)),
    fail. % -- No continuar con los demás síntomas de una enfermedad

/* Justificar si el paciente desea saber el porque de la pregunta */
procesar_rpta(Diagnosis, Sintoma, Rpta) :-
    Rpta == porque,
    write('-----'),nl,
    write('Se está analizando la hipotesis '),nl,
    write('que el paciente pueda tener: '),
    write(Diagnosis), write('. '), nl,
    write('Por tanto, necesito saber si el paciente tiene '),nl,
    write(Sintoma), write('. '), nl,
    write('-----'),nl,
    write('Entonces, por favor responda'), nl,
    pregunta_a_paciente(Diagnosis, Sintoma).

/* Efectuar pregunta sobre el síntoma y procesar respuesta */
pregunta_a_paciente(Diagnosis, Sintoma) :-
    write('El paciente tiene '),
    write(Sintoma),
    write('? '),
    read(Rpta),
    procesar_rpta(Diagnosis, Sintoma, Rpta).

/* Si la pregunta sobre el síntoma ya fue contestada
afirmativamente o negativamente, ya no efectuar
la pregunta */
pregunta(_, Sintoma) :-
    respuestas_afirmativas(Sintoma).

pregunta(_, Sintoma) :-
    respuestas_negativas(Sintoma),
    fail.

/* Si no se respondió con anterioridad sobre el síntoma,
efectuar pregunta sobre el síntoma */
pregunta(Diagnosis, Sintoma) :-
    pregunta_a_paciente(Diagnosis, Sintoma).

/* Termina de efectuar preguntas cuando la lista está vacía*/
preguntar(_, []).
/* Efectúa el proceso de preguntar de manera recursiva */
preguntar(Diagnosis, [Head|Tail]) :-
    pregunta(Diagnosis, Head), /* Procesa primera pregunta de la lista */
    preguntar(Diagnosis, Tail). /* Procesa recursivamente el resto de preguntas */

```

```

/* Fase 3: Efectuar el diagnóstico respectivo */
mostrar_sintomas([]).
mostrar_sintomas([Head|Tail]) :-
    write('- '),write(Head),nl,
    mostrar_sintomas(Tail).

diagnostico(Diagnosis, Sintomas) :-
    write('*****'),nl,
    write('El paciente tiene '),
    write(Diagnosis),nl,
    write('porque presenta los siguientes síntomas:'),nl,
    mostrar_sintomas(Sintomas).

/* ***** MÓDULO PRINCIPAL ***** */
consultar() :-
    /* Fase 0: Inicializar variables dinámicas y mensajes previos */
    retractall(respuestas_afirmativas(_)),
    retractall(respuestas_negativas(_)),
    mensajes_previos,
    /* Fase 1: Recuperar enfermedad y síntomas*/
    sintomas(Diagnosis, Sintomas),
    /* Fase 2: Efectuar preguntas al paciente*/
    preguntar(Diagnosis, Sintomas),
    /* Fase 3: Efectuar el diagnóstico respectivo */
    diagnostico(Diagnosis, Sintomas).

```

- Almacenar, compilar y ejecutar.
- Probar para diferentes enfermedades, suministrando diferentes síntomas.

5.- EJERCICIOS COMPLEMENTARIOS.

1. Implementar un sistema experto básico (tema libre).

Nota.- Presentar la solución de estos ejercicios en la siguiente sesión de laboratorio.